

Chap.6 YOLO (You Only Look Once)

방 수 식 교수
(bang@tukorea.ac.kr)

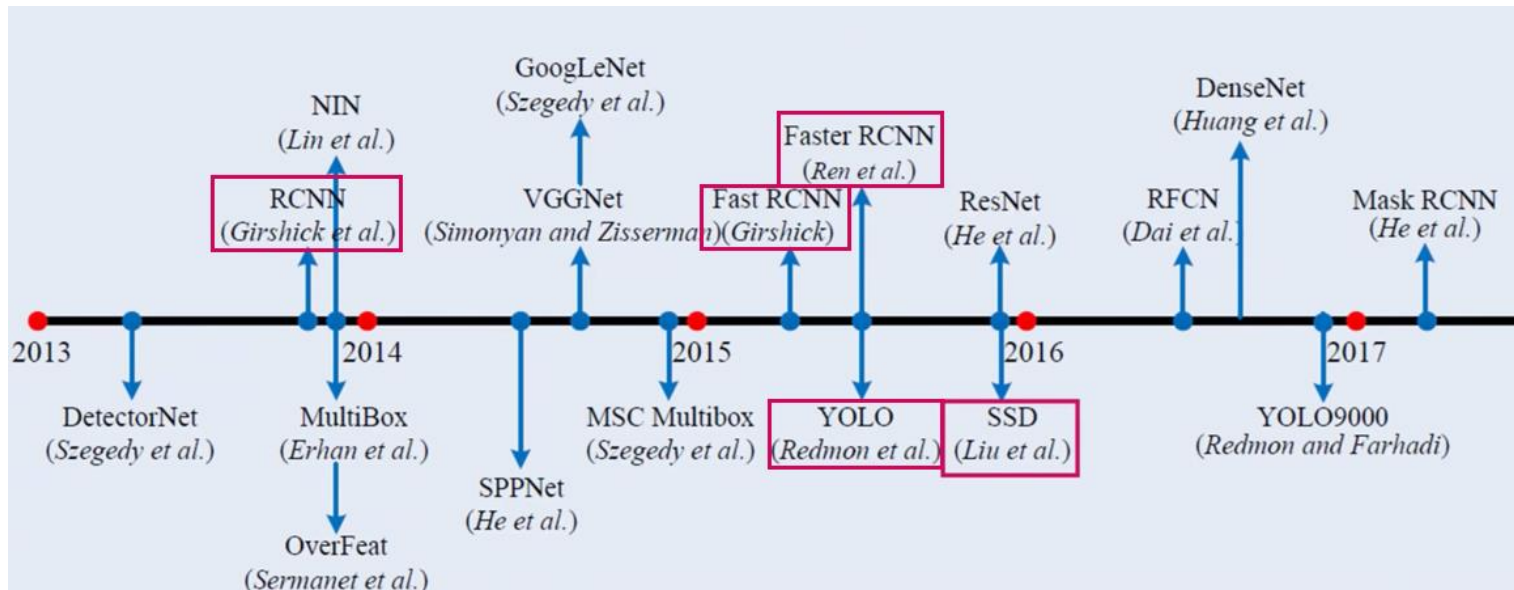
한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

Object Detection (객체 검출)



- CNN 알고리즘을 활용하여 다양한 분야에서 활용됨
- 2-stage Detector와 1-stage Detector로 분류
 - 2-stage Detector는 전체 task를 두 단계로 나누어 순차적 실행 (R-CNN 계열)
 - 1-stage Detector는 두 단계를 동시에 실행 (YOLO계열)



2-stage Detector 처리 속도 ↓, 정확도 ↑
1-stage Detector 처리 속도 ↑, 정확도 ↓

Limitation of R-CNN



■ 속도가 느림

- R-CNN은 CNN을 실행하기 전 Selective Search 과정을 거치기 때문

■ Background error 잦음

- 배경에 객체가 있다고 판단하는 오류
- R-CNN은 Selective Search 알고리즘을 통해 제안된 영역만을 사용하기 때문에 배경을 객체로 탐지하는 오류 발생

■ Crop시 이미지 왜곡 발생

- 이미지를 crop하는 과정에서 형태가 일그러져서 분류성능 저하



해당 문제들을 해결하기 위해 YOLO 탄생

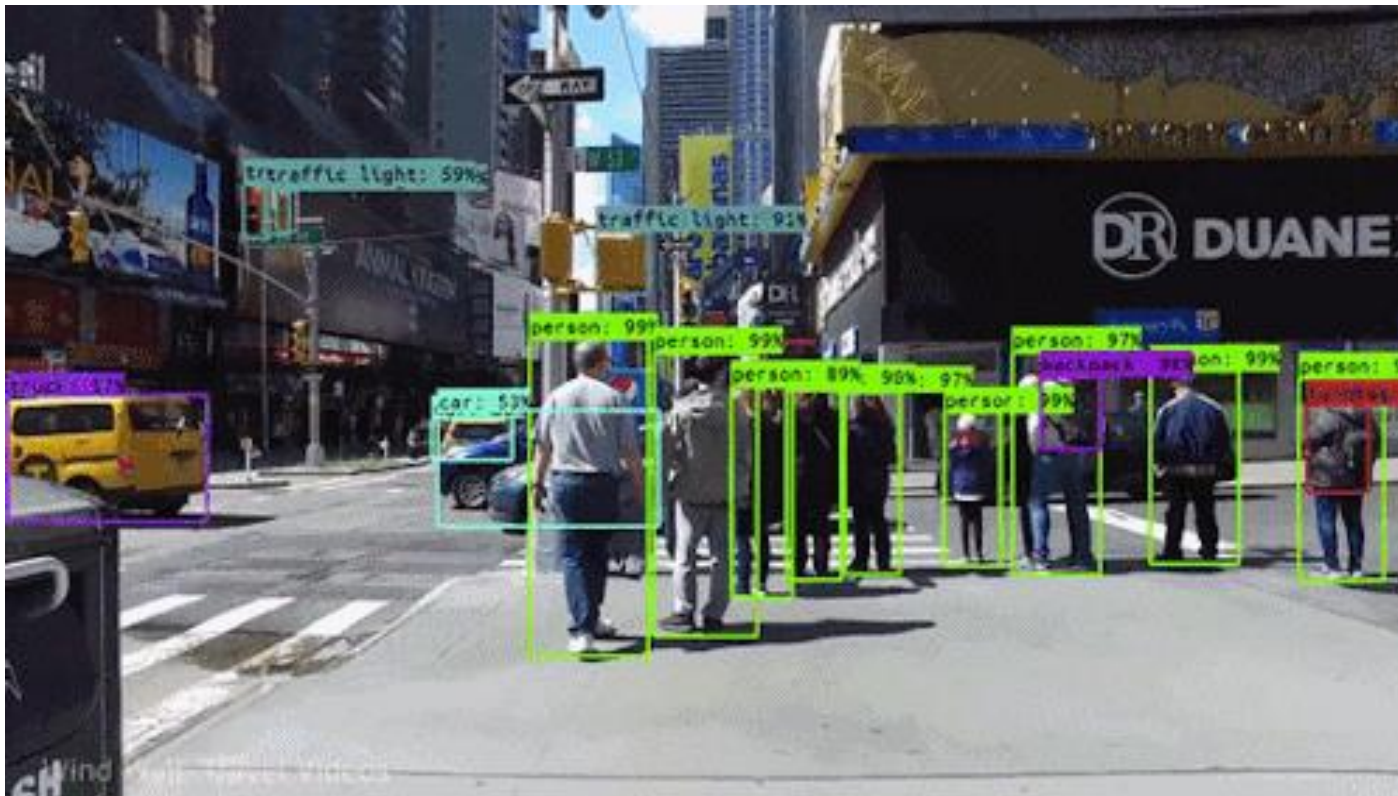
1. YOLO는 **한번의 Neural Network 실행** 만으로 해결됨. (1-stage Detector)
2. YOLO는 이미지 전체를 이용하여 예측하기 때문에 **Background error가 작음.**
3. YOLO는 이미지 전체를 이용하여 예측하기 때문에 **왜곡이 없음.**

YOLO (You Only Look Once)



■ YOLO (You Only Look Once)

- “한번만 보겠다” 속도가 빠르다는 장점을 내세운 Object Detection 모델
- **Localization**과 **Classification**을 동시에 진행되도록 설계
- 대표적인 **1-stage Detector**의 모델 중 하나

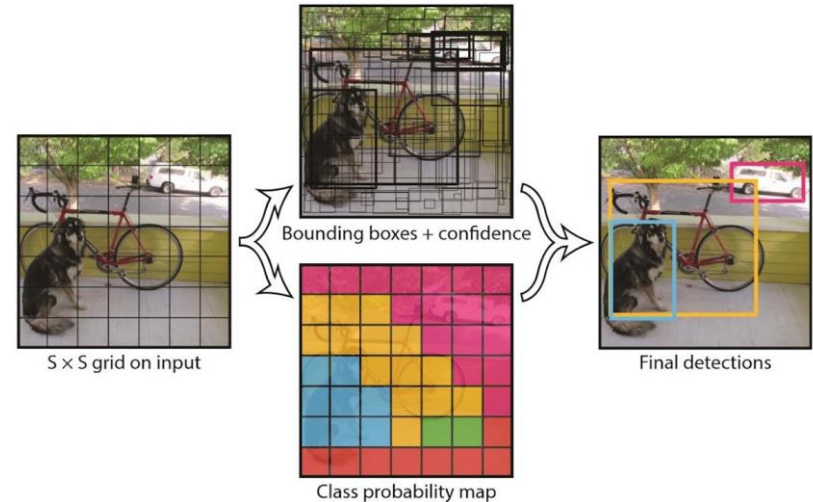


YOLO (You Only Look Once)



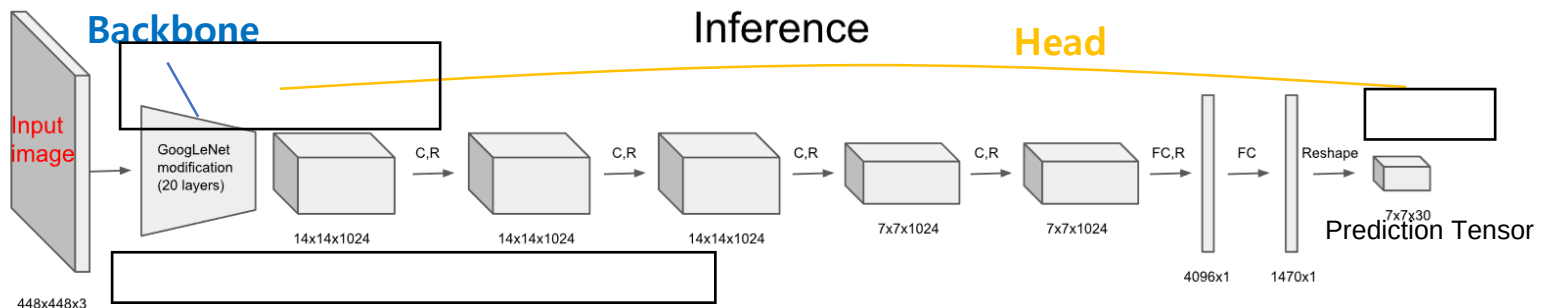
■ YOLO 수행 과정

1. 이미지를 $S \times S$ 의 Grid로 분할
 2. 이미지 전체를 신경망에 넣고
특징 추출을 통해 Prediction Tensor 생성
 3. Grid 별 예측 정보를 바탕으로
Box 조정 및 분류 수행
- Tensor: 다차원 배열



■ YOLO network design

- YOLO는 Backbone과 Head구조로 구성
- Pretrain된 20개의 Convolutional Layer(Backbone) 뒤에
4개의 Convolutional Layer와 2개의 Fully Connected Layer(Head)를 추가

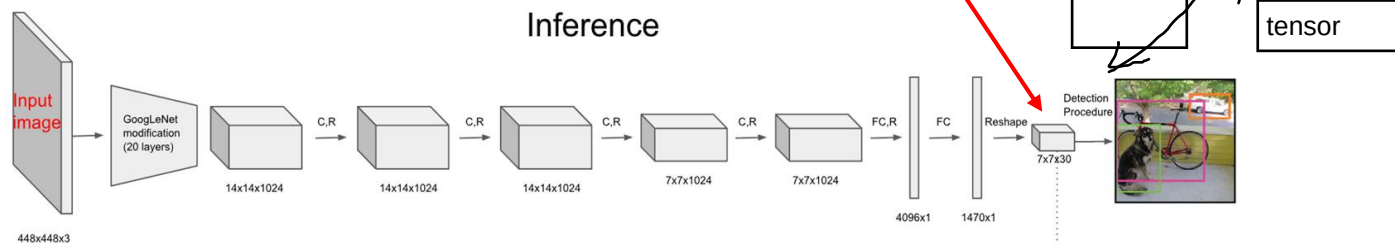


Inference Process

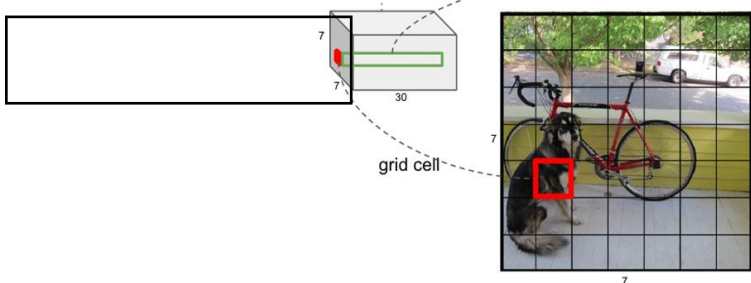


■ Inference Process (논문 기준)

- S : 이미지를 S X S Grid로 나눔
- B : 하나의 Grid Cell에서 B 개의 Bounding Box 예측
- B에 대한 예측치는 총 5개 => x, y, w, h, c (뒷 장에서 설명)
- 이 경우 **S by S by (B x 5 + C)**의 Tensor 생성
- S = 7, B = 2, C = 클래스 수 (20으로 가정) → **(7 by 7 by 30)**



Tensor values interpretation



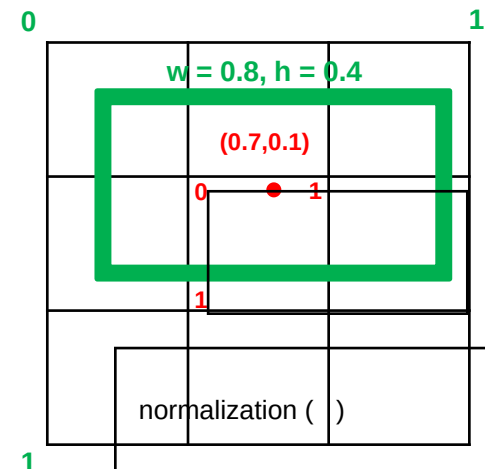
(x,y,w,h,c),(x,y,w,h,c),(0,0,0,...)

Prediction Tensor

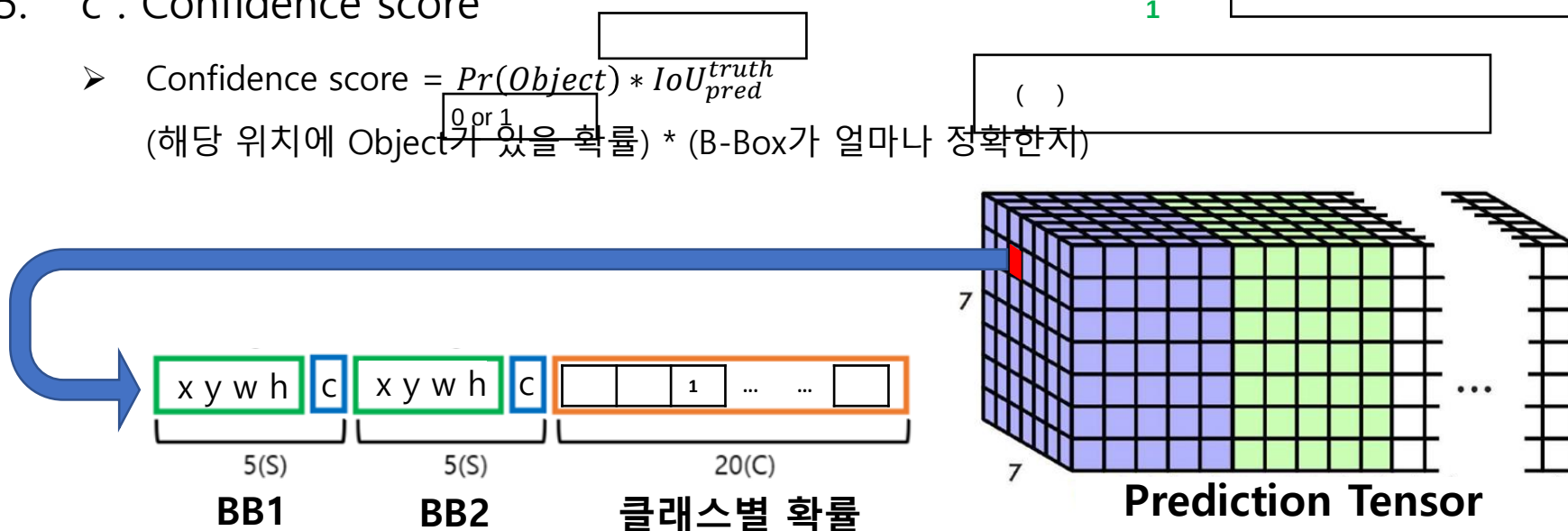


■ B의 예측치 5종

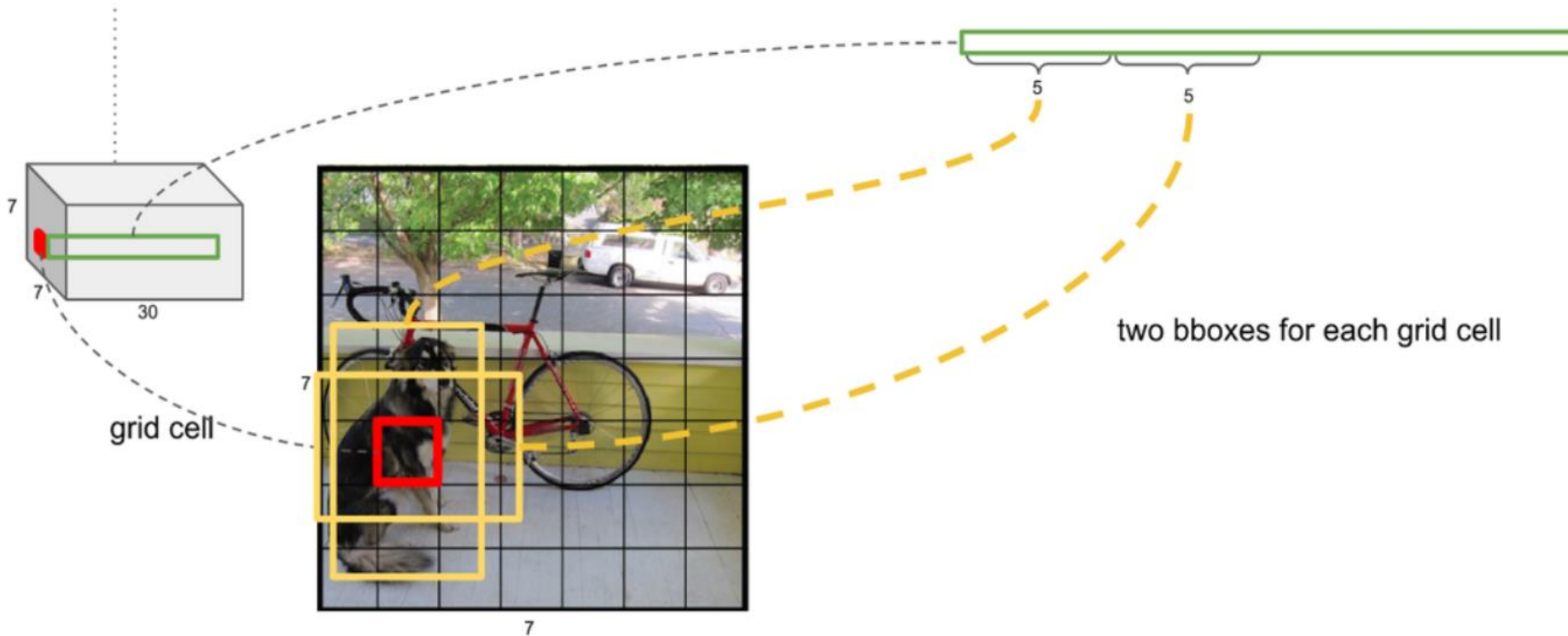
1. x : 해당 Grid Cell 에서 해당 BB(Bounding Box) 중앙의 x 좌표 (0~1)
2. y : 해당 Grid Cell 에서 해당 BB 중앙의 y 좌표 (0~1)
 - (x,y) 가 정확히 Grid Cell의 정 중앙에 위치한다면 (0.5, 0.5)
3. w : 전체 이미지 크기에 대한 BB의 너비 (0~1)
4. h : 전체 이미지 크기에 대한 BB의 높이 (0~1)
 - 이미지 전체의 크기를 (1,1) 이라고 했을 때 BB의 상대적인 크기
5. c : Confidence score



- Confidence score = $Pr(Object) * IoU_{pred}^{truth}$
(해당 위치에 Object가 있을 확률) * (B-Box가 얼마나 정확한지)

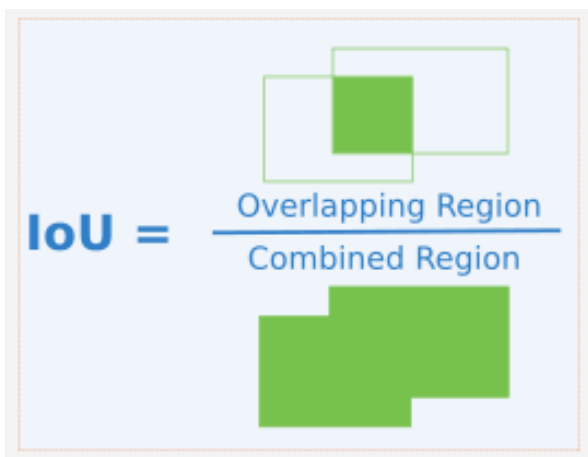


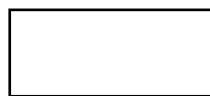
Prediction Tensor



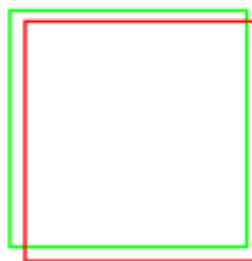
■ IoU (Intersection over Union)

- Bounding Box에 널리 사용되는 지표
- 두 Bounding Box가 얼마나 겹치는지에 한 지표
- 두개의 Bounding Box 사이에 중첩되는 영역을 전체 영역으로 나눈 것

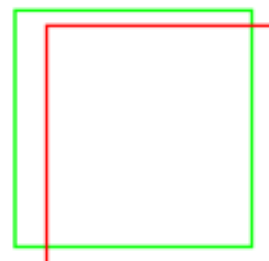

$$\text{IoU} = \frac{\text{Overlapping Region}}{\text{Combined Region}}$$



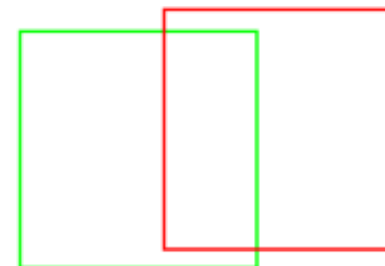
IoU=0.92



IoU=0.71



IoU=0.39



IoU = 교집합 영역 넓이 / 합집합 영역 넓이

Training Image → Target Tensor 준비



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.



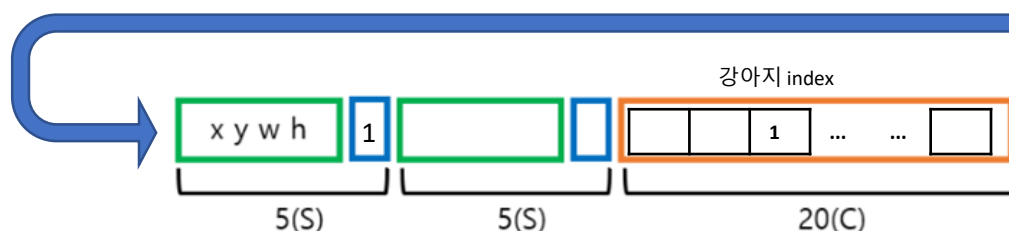
Target Tensor



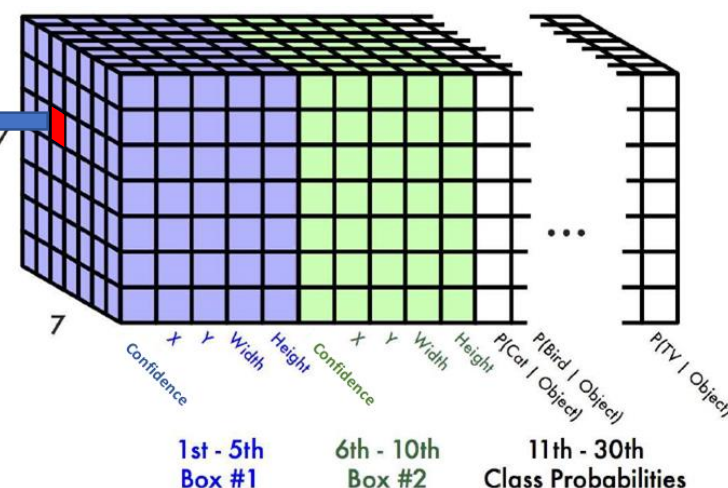
■ 학습을 위한 Target Tensor 생성

- YOLO의 Output과 동일한 형태의 $7*7*30$ Target 생성 (20 classes 기준)
- $7*7*30$ 의 0으로 초기화된 배열을 생성
- Label Box가 몇 번째 Grid에 있는지 파악
- 해당 Grid의 배열에 정답값 입력

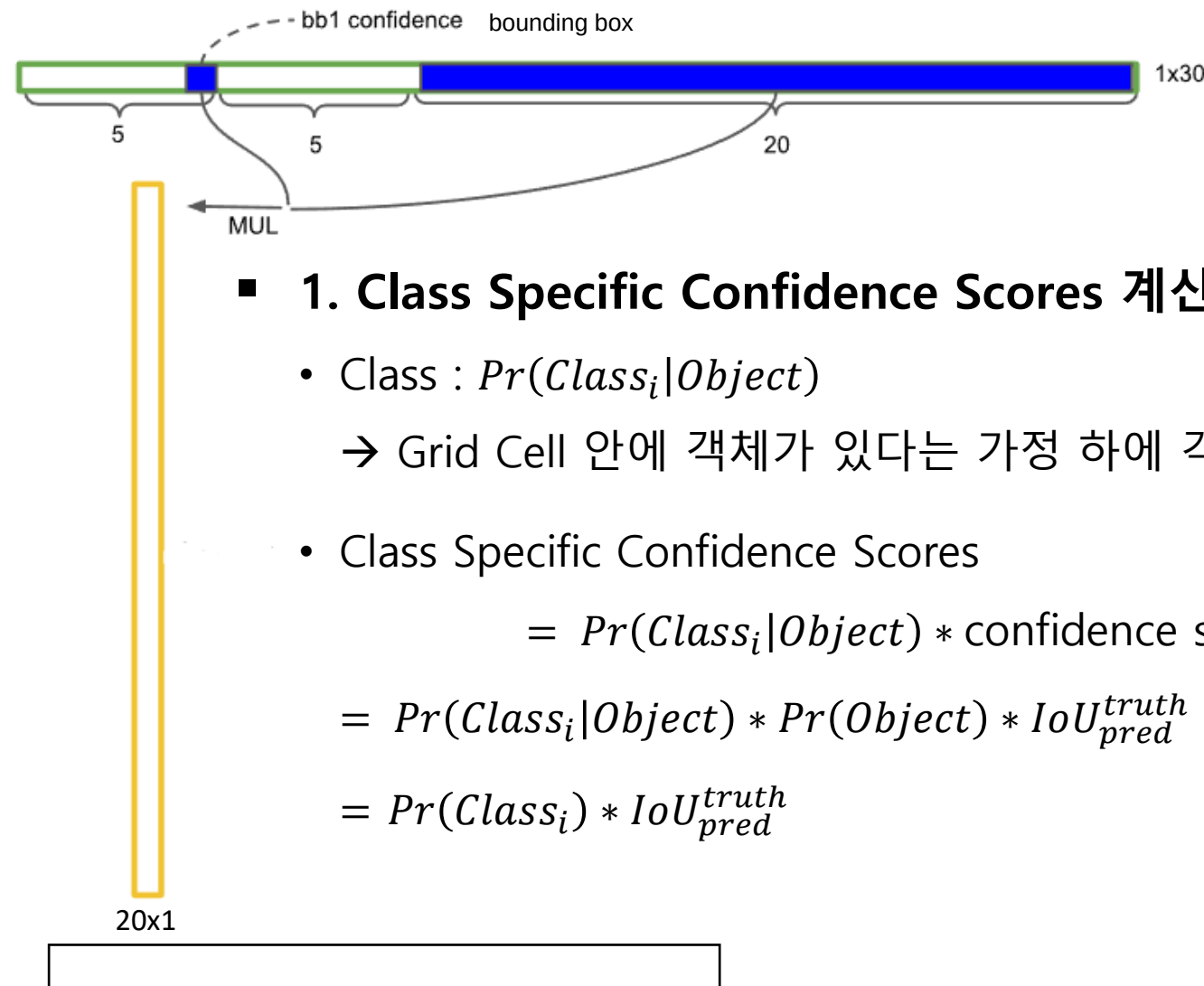
강아지가 (2,2) 그리드에 있다면?



나머지 공간은 처음에 초기화한 대로 0



Objection Detection 과정 1



1. Class Specific Confidence Scores 계산

- Class : $Pr(Class_i|Object)$

→ Grid Cell 안에 객체가 있다는 가정 하에 각 Class에 대한 확률

- Class Specific Confidence Scores

$$= Pr(Class_i|Object) * \text{confidence score}$$

$$= Pr(Class_i|Object) * Pr(Object) * IoU_{pred}^{truth}$$

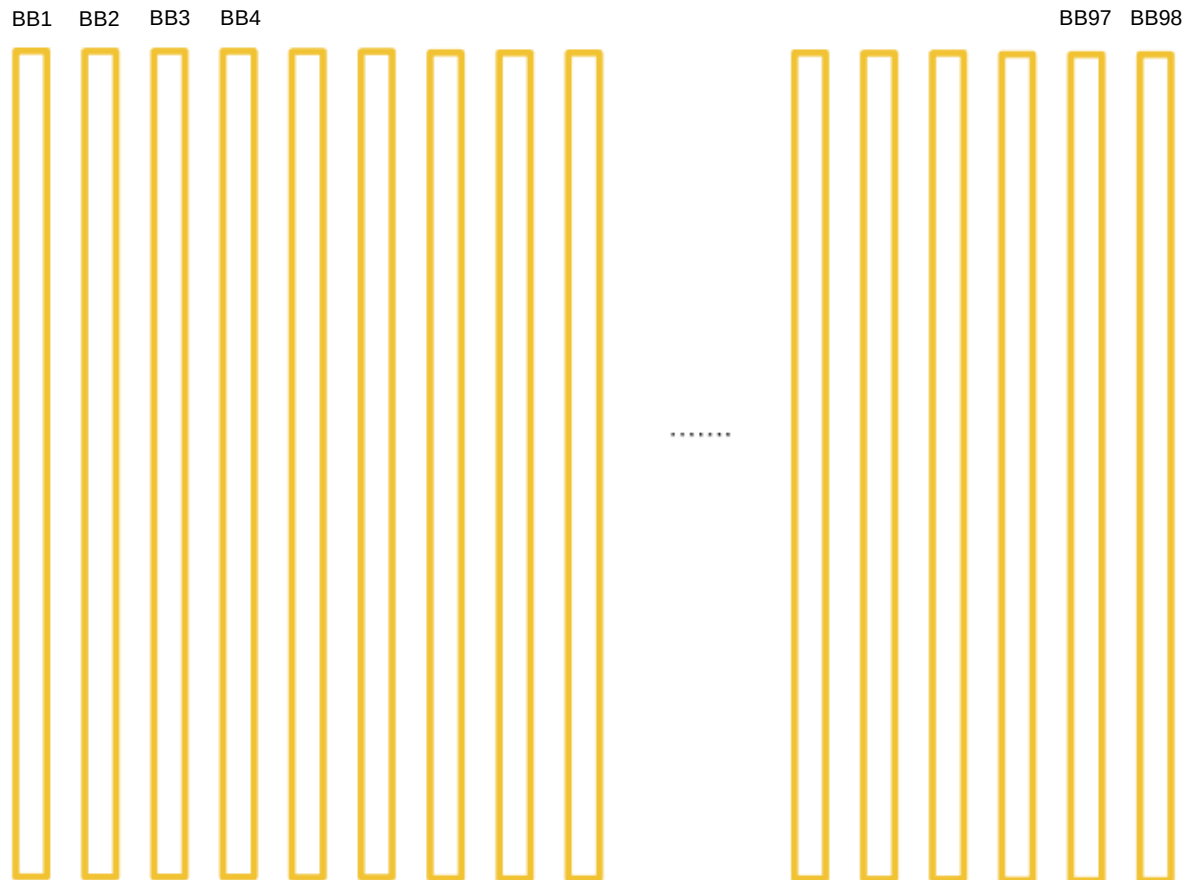
$$= Pr(Class_i) * IoU_{pred}^{truth}$$

Objection Detection 과정 1



■ 1. Class Specific Confidence Score

- 해당 과정을 모든 Grid Cell의 BB에 대해 수행
- 총 98개의 Class Specific Confidence Score를 얻음



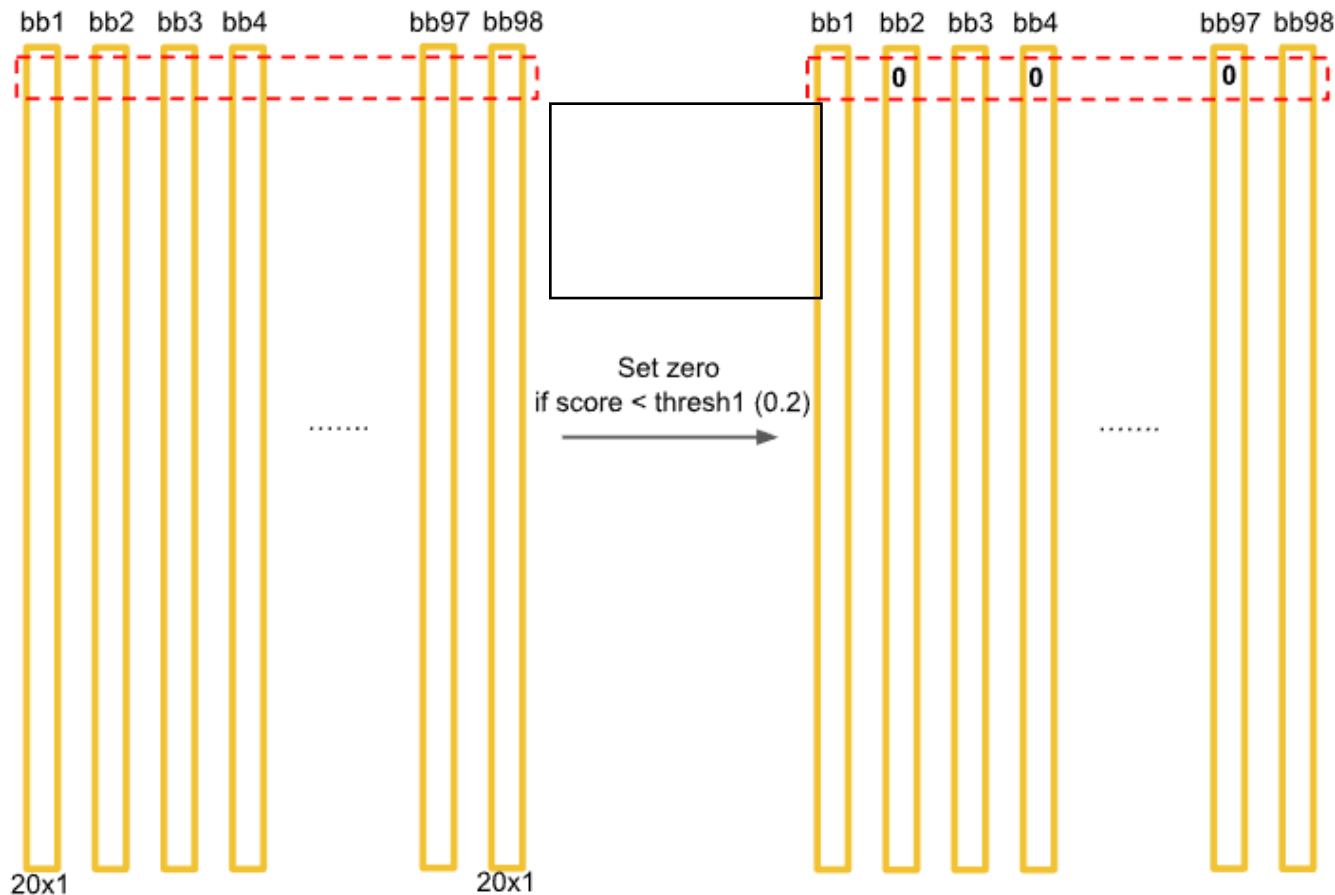
Total $7*7*2=98$ 개의 BB

Objection Detection 과정 2



2. Threshold 적용 (0번째 클래스 예시)

- 각 BB에서 0번째 클래스 score 확인
- Threshold 보다 낮은 score는 0으로 설정 ex) Threshold : 0.2인 경우



- Threshold 미적용 시

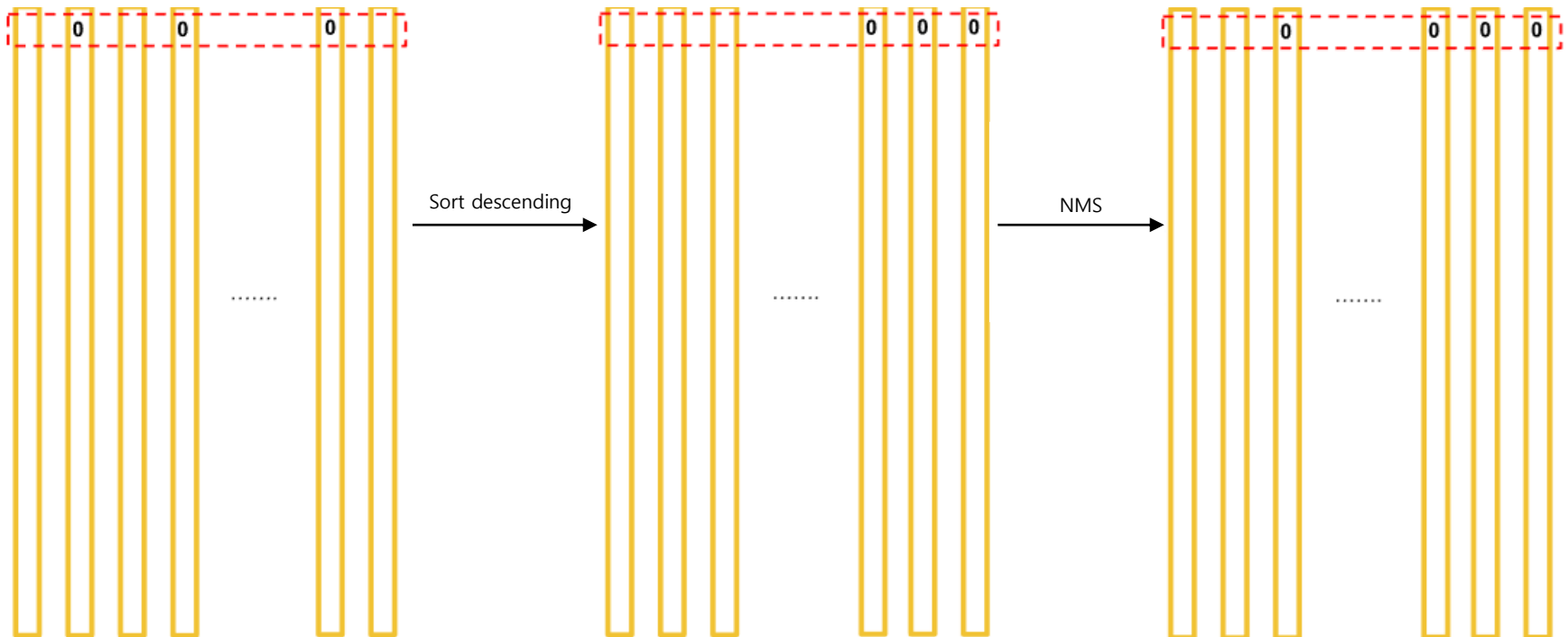


Objection Detection 과정 3



■ 3. NMS 수행 (서로 중복되는 Box 제거)

- 점수가 높은 순서로 정렬
- 불필요한 BB들을 처리하기 위해 **NMS(Non – Maximum Suppression)** 수행
→ NMS를 시행 시, 같은 Object를 가리키는 BB를 제거



Objection Detection 과정 3

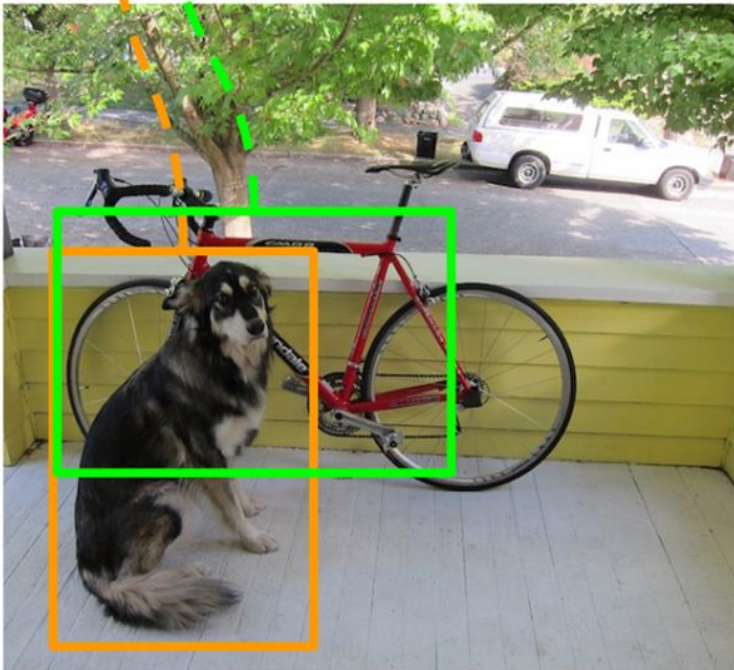


3. NMS(Non – Maximum Suppression)

Class (dog) scores for each bb

bb47 bb20 bb15 bb7										bb1 bb4 bb8 bb98			
0.5	0.3	0.2	0.1							0	0	0	0

1x98



NMS 수행 과정 (예시)

- 0번째 Class(Dog)에 대해서 Class Specific Confidence Score를 내림차순으로 정렬
- 가장 높은 점수인 **BB47**와 두번째로 높은 점수인 **BB20**의 **IoU** 계산
- $\text{IoU}(\text{BB47}, \text{BB20}) > 0.5$ 이면 **BB20** 의 **score**를 **0**으로 설정
- 이후, 다음 BB와 해당 과정 진행

중복
제거

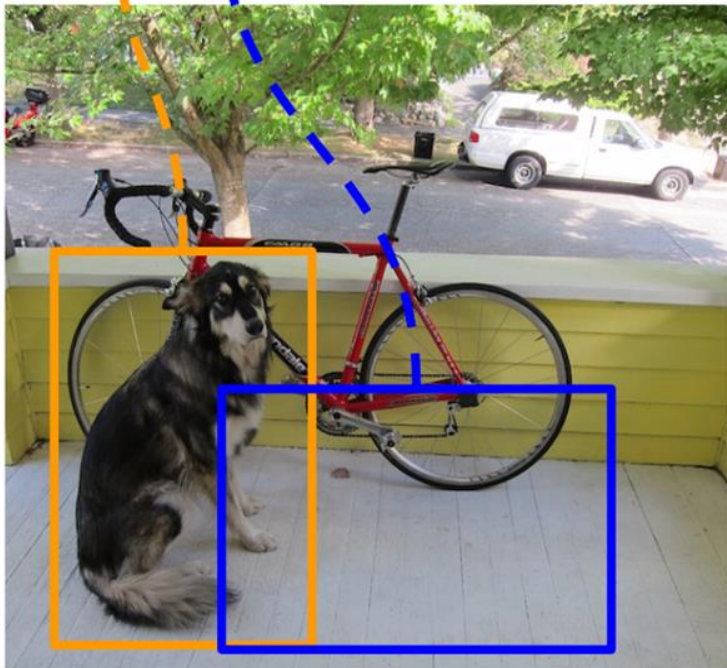
Objection Detection 과정 3



3. NMS(Non – Maximum Suppression)

class (dog) scores for each bbox

										bb1	bb4	bb8	bb98	
bb47 bb20 bb15 bb7														
class: dog	0.5	0	0.2	0.1						0	0	0	0	1x98



■ NMS 수행 과정

1. 가장 높은 점수인 **BB47**와
두번째로 높은 점수인 **BB15**의 **IoU** 계산
2. $\text{IoU}(\text{BB47}, \text{BB15}) < 0.5$ 이면
BB15의 **score** 유지
3. 이후, score가 0이 아닌 모든 BB와 진행

Objection Detection 과정 3

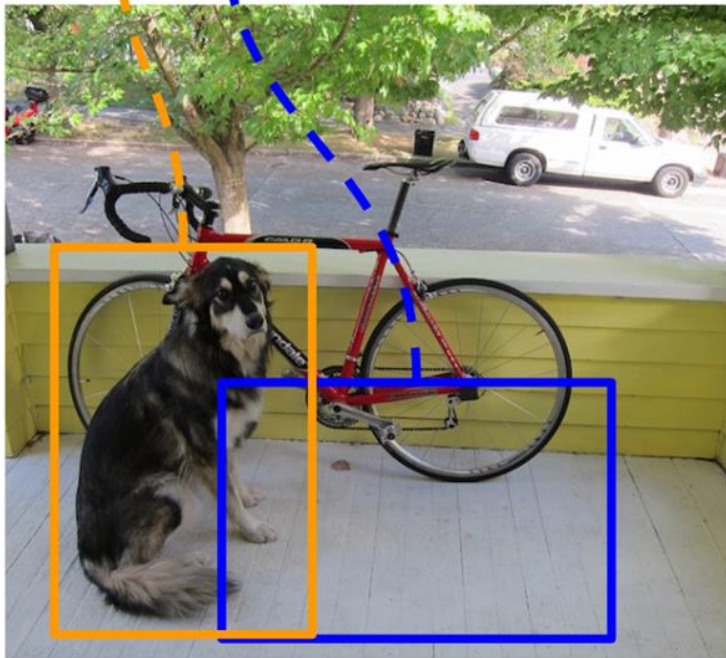


3. NMS(Non – Maximum Suppression)

class (dog) scores for each bbox

bb47 bb20 bb15 bb7										bb1 bb4 bb8 bb98			
0.5	0	0.2	0							0	0	0	0

1x98



■ NMS 수행 과정

1. 모든 BB에 대해 NMS를 진행하면

Class(dog)가 종료

➤ 대부분의 값들은 0이 되고,
가장 높은 score를 가진 BB만 남게 됨.

2. 이후, **다른 모든 Class**에 대해 NMS 진행

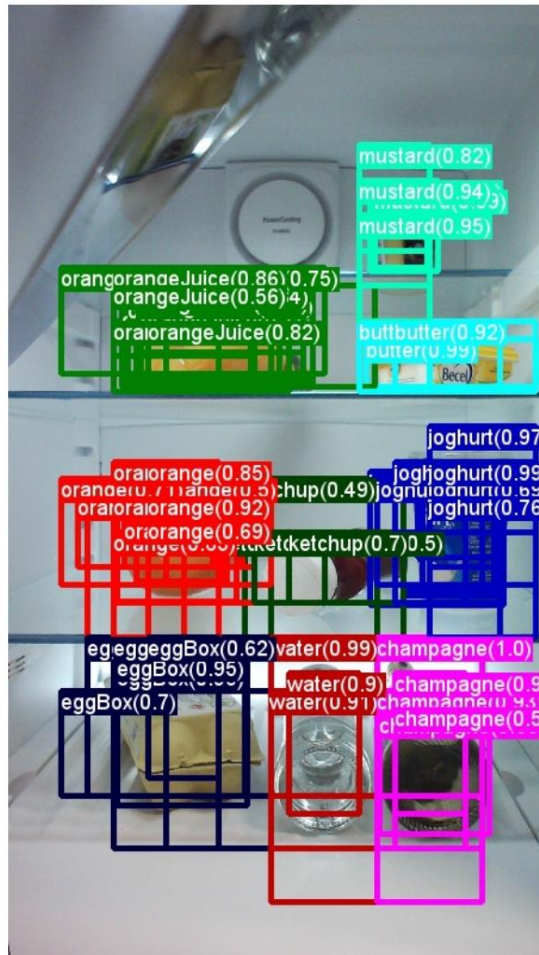
Objection Detection 과정 3



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

3. NMS 수행 (서로 중복되는 Box 제거)

NMS 미적용



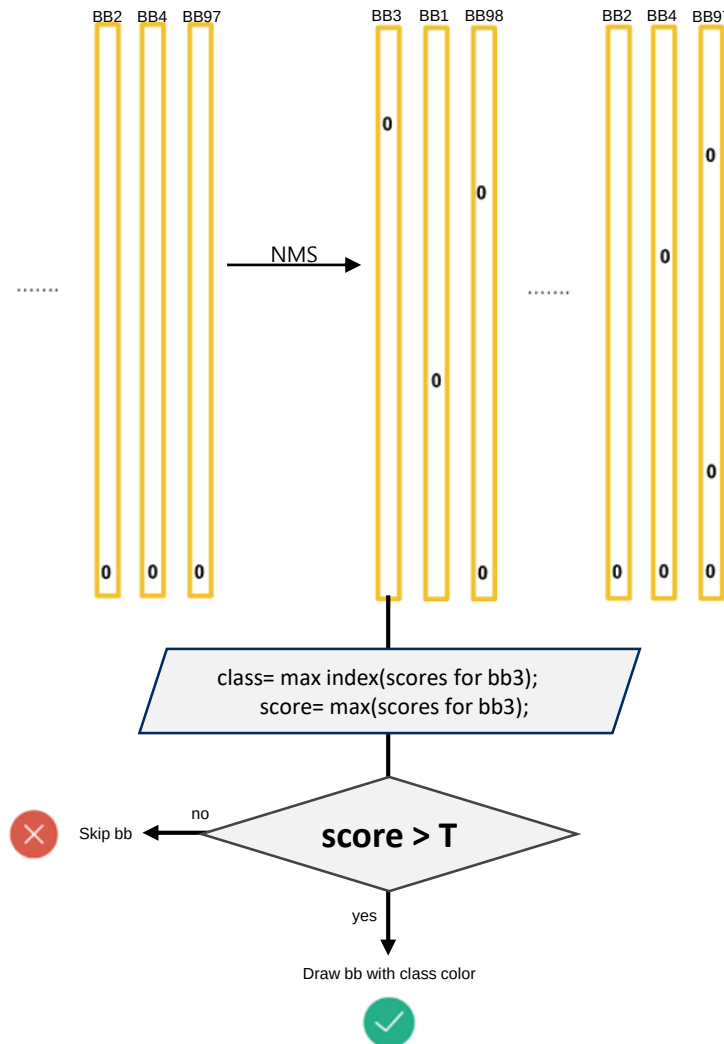
NMS 적용



Objection Detection 과정 4



Box 표현



- 가장 높은 Score를 해당 Class로 매칭 및 박스 표현

Ex)

	BB3
강아지	0
고양이	0.1
비행기	0.1
자동차	0
자전거	0.7
표지판	0.05
핸드폰	0.05

Class = 자전거
Score = 0.7

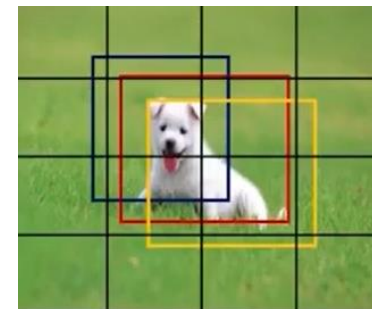
Training Process (YOLO v1 기준)



■ Loss function의 설명 전 몇가지 전제조건

- Inference 단계에서는 Grid cell당 여러 BB 를 예측하지만, Training단계에서는 하나의 BB 만 사용.

(Ground-truth box 과 IOU 높은 BB 로)



■ Ground-truth
■ b-box1
■ b-box2

- Sum Squared Error(SSE)를 이용해 손실 계산
- 3가지 Loss에 대한 합을 전체 Loss로 지정
 1. The localization loss (예측된 BB의 위치 및 크기 오류 계산)
 2. The confidence loss (예측된 BB의 개체 유무 확인)
 3. The classification loss(예측된 BB의 개체가 속한 class)

Training Process (YOLO v1 기준)



i = cell index, j = B-Box predictor

$\mathbb{1}_{ij}^{obj}$: Object가 존재하는 grid cell i 의 predictor B-Box j
 $\mathbb{1}_{ij}^{noobj}$: Object가 존재하지 않는 grid cell i 의 B-Box j } 0 or 1(선택된 B-Box)

λ_{coord} : 5 (b-box coordinates loss) λ_{noobj} : 0.5 (no object의 class probability loss)

5 ()

1. The localization loss (예측된 b- box의 위치 및 크기 오류 계산)

$$\textcircled{1} \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

x - coordinate of bbox center inside cell
 y - coordinate of bbox center inside cell
 w - bbox width ([0; 1] wrt image)
 h - bbox height ([0; 1] wrt image)

$$\textcircled{2} \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

큰 Box보다 작은 box가 위치변화에 민감 -> w, h 에 square root 사용

Training Process (YOLO v1 기준)



2. The confidence loss (예측된 b- box의 개체 유무 확인)

$$\textcircled{3} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 \text{ (Object detect)}$$

$$\textcircled{4} \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \text{ (No Object detect)}$$

$$C_i : Pr(Object) * IOU_{pred}^{truth}$$
$$C_i = \begin{cases} 1, & \text{Object detect} \\ 0, & \text{No Object detect} \end{cases}$$

3. The classification loss (예측된 b- box의 개체가 속한 class)

$$\textcircled{5} \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

$$p_i(c) : Pr(Class_i | Object)$$

각 Cell의 loss는 각 class에 대한 class 조건부 확률의 오차제곱

Loss Function

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad \textcircled{1}$$

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad \textcircled{2}$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \quad \textcircled{3}$$

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \quad \textcircled{4}$$

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad \textcircled{5}$$

■ Limitation of YOLOv1

• 작은 객체 검출 정확도가 낮음

- YOLOv1은 이미지를 그리드로 나누고 그리드 내에 있는 객체를 예측하기 때문에, 작은 객체를 검출하기 어려움

• 겹쳐진 객체 검출이 어려움

- YOLOv1은 각 그리드 셀에 대해 하나의 객체만을 예측
- 따라서 객체들이 붙어있을 경우 겹친 객체를 분리하기 어려움

• 부정확한 Bounding Box

- 몇 단계의 layer를 거쳐서 나온 feature map을 대상으로 Bounding Box를 예측하기 때문에, localization이 다소 부정확할 수 있음

YOLO version 비교



■ YOLOv2

- 기존 YOLOv1의 속도 유지 및 정확도 개선
- YOLOv1의 문제점인 작은 물체 탐지 및 세밀한 classification 가능

■ YOLOv3

- YOLOv2의 정확도 개선, but 속도 저하
- YOLOv2에 비해 10배 더 많은 Bounding Box 예측

■ YOLOv4

- YOLOv3의 정확도와 속도 모두를 개선
- 연산량을 줄이는 것이 아닌, 병렬 계산이 가능하게 하여 속도 최적화

■ YOLOv5

- YOLOv4의 속도와 정확도를 개성
- YOLOv4 보다 메모리 사용량이 적으며 학습 시간도 비약적으로 줄임
- 최적화가 잘 되어있어 weight file의 크기가 작고, 다양한 크기의 network를 상황에 맞게 선택 가능

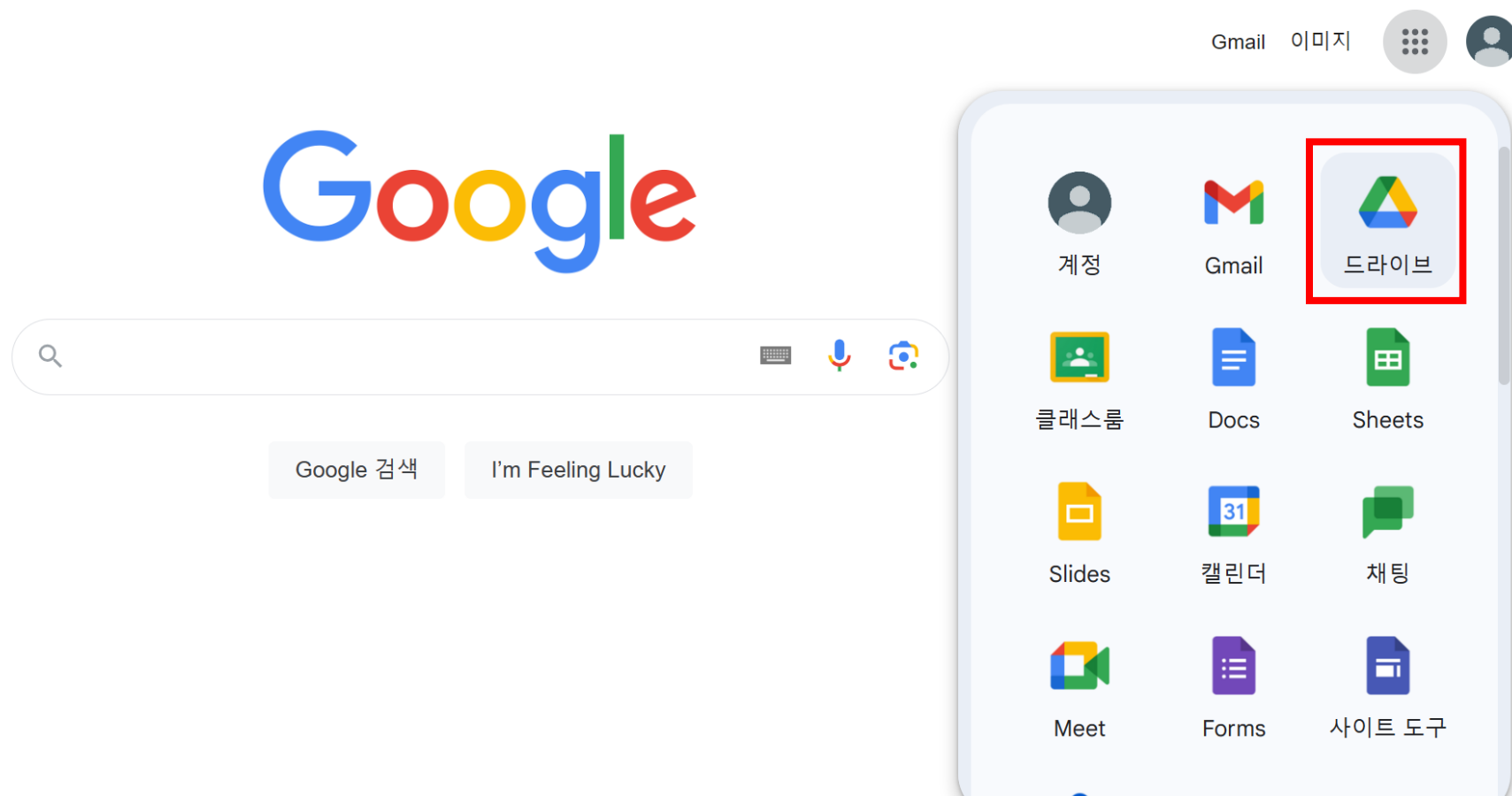
YOLOv5 실습



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

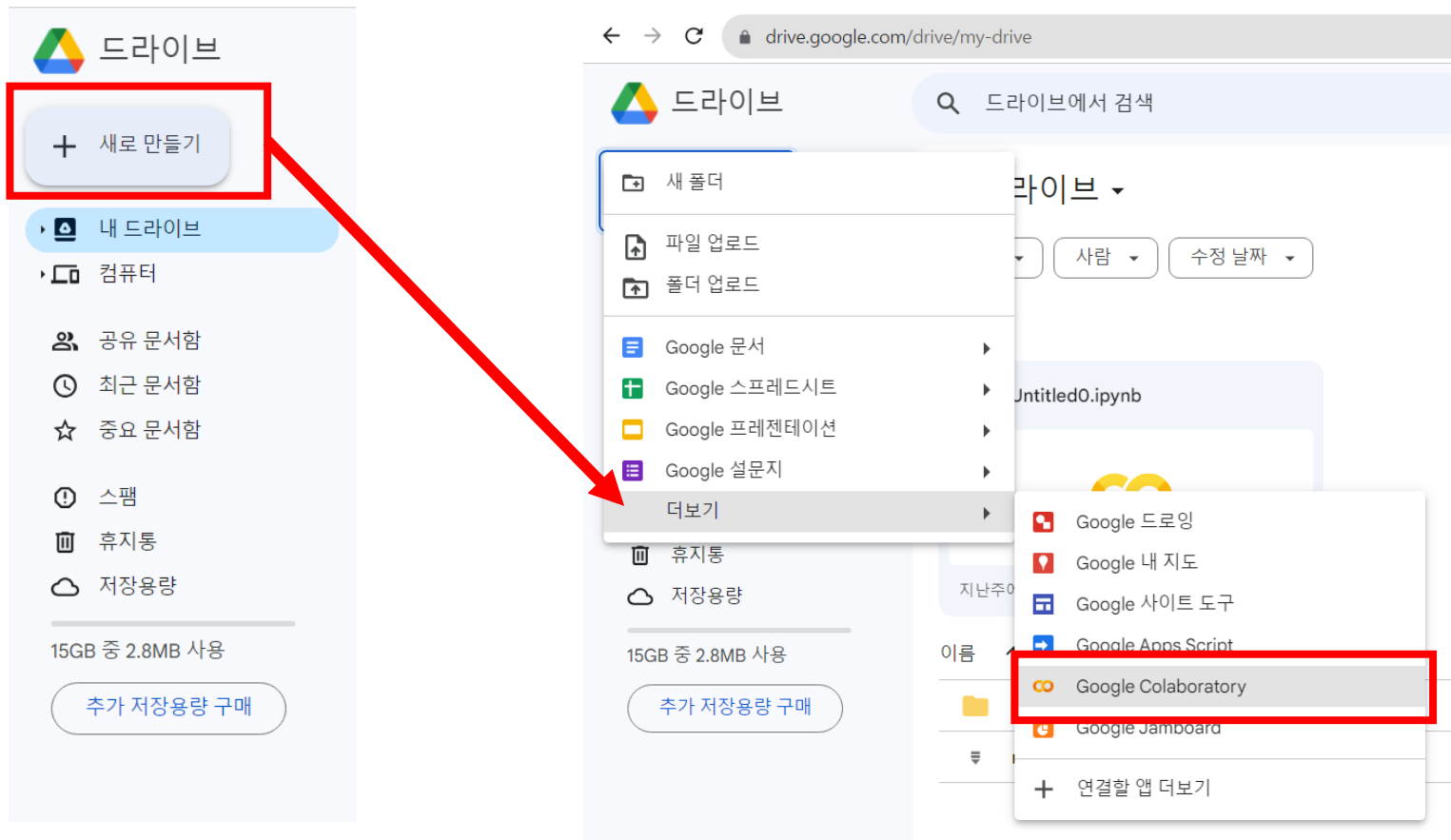
■ 구글 드라이브에 MNIST 데이터 업로드

- 구글 로그인 후 드라이브 선택
- 드라이브에 R-CNN에서 사용했던 MNIST 데이터 zip파일 업로드



■ Colab 실행 방법

- 새로 만들기 클릭 후, 더보기에서 Google Colaboratory 선택



YOLOv5 실습

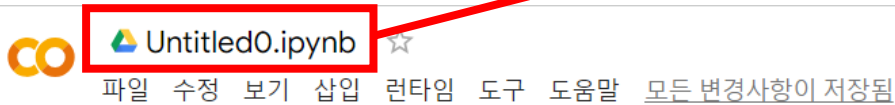


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ Colab 실행 방법

• 기본 사용법

클릭하여 파일 이름 변경 가능



코드 셀 추가 가능

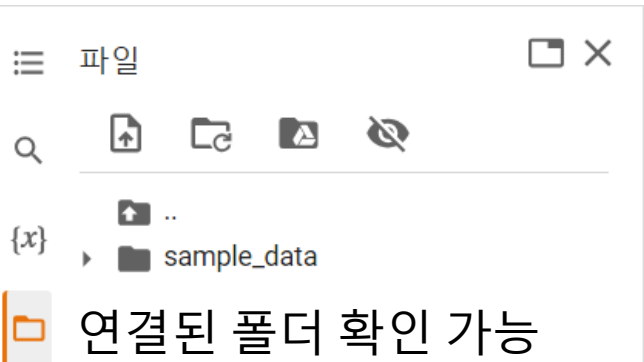


셀 실행 버튼 (단축키 : shift + enter / ctrl + enter)



변수 목록 확인 가능

{x}



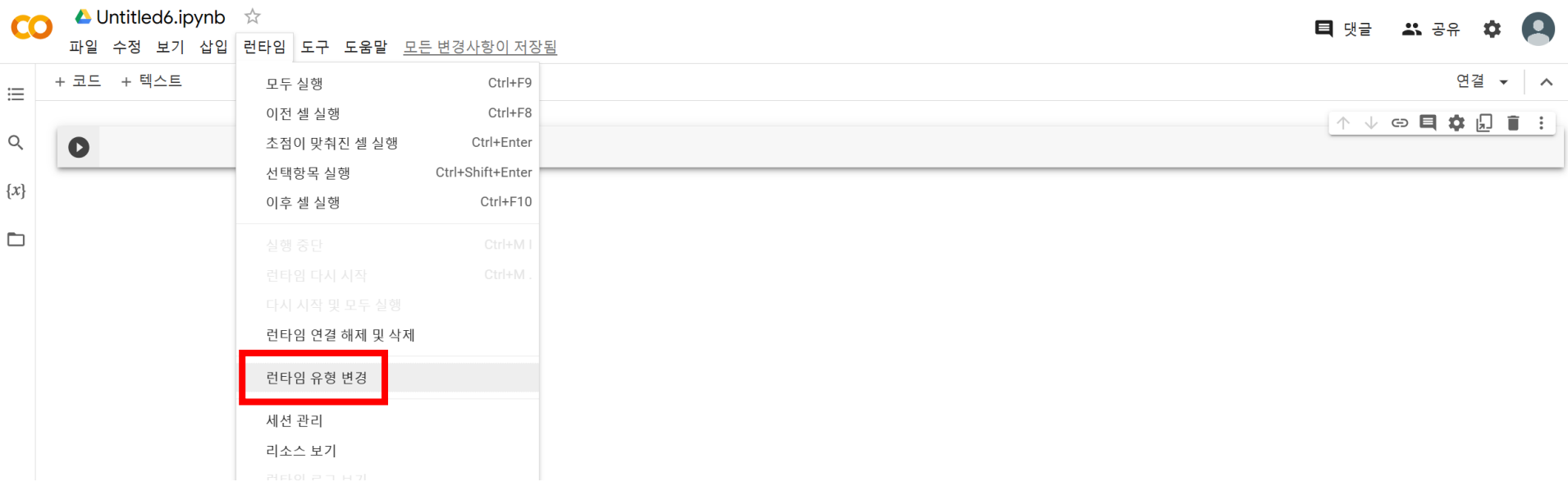
연결된 폴더 확인 가능

YOLOv5 실습



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

- Colab 실행 방법 – GPU 설정
 - 런타임 유형 변경 선택



■ Colab 실행 방법 – GPU 설정

- T4 GPU 선택 후 저장

런타임 유형 변경

런타임 유형

Python 3

하드웨어 가속기 ?



CPU



T4 GPU



A100 GPU



V100 GPU



TPU

프리미엄 GPU를 이용하실까요? [추가 컴퓨팅 단위 구매](#)

취소

저장

■ 드라이브 연결

- 구글 드라이브를 colab과 연결

```
from google.colab import drive  
drive.mount('/content/drive') #구글 드라이브 연결
```

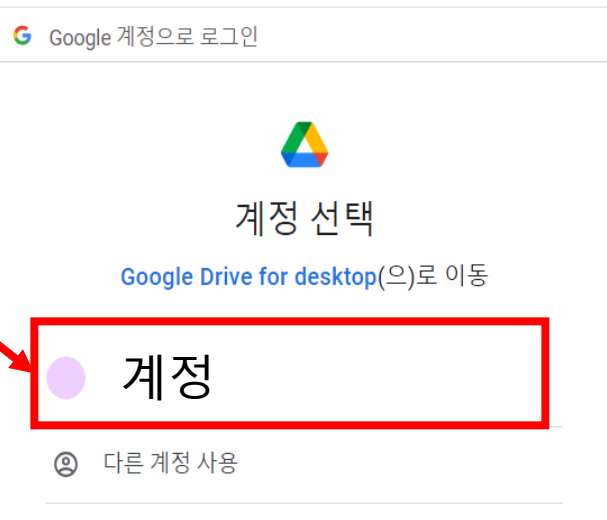
Mounted at /content/drive

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

[Google Drive에 연결](#)



Google Drive에 연결 -> 계정선택 -> 허용

■ 실습 데이터 준비

- 현재 내 경로를 확인하는 방법

```
!pwd
```

pwd : printing working directory

“!” : 느낌표 뒤에 커널 명령어 입력 가능

```
/content
```

- MNIST 데이터를 업로드한 드라이브 폴더로 경로 변경

▼ YOLO_practice

■ MNIST.zip



경로 복사 가능

```
%cd 복사한 경로 바로 입력
```

cd : change directory

“%” : cd 앞에 %를 붙이는 경우,
변경된 경로가 모든 셀에서 적용됨

- 데이터 압축 해제

```
!unzip -qq "MNIST.zip" #압축 해제
```



▼ YOLO_practice

▶ data

■ MNIST.zip

■ data.yaml

압축 해제가 되어있는 경우, 실행 x

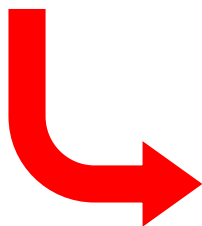
■ 실습 데이터 준비

- 파일 리스트 불러오기 및 이미지 개수 확인

```
from glob import glob  
  
img_list = glob("data/images/*.jpg") #해당 폴더에 있는 파일 리스트
```

- train set, validation set 분할

```
from sklearn.model_selection import train_test_split  
  
#학습 시간 단축을 위해 train image는 조금만 사용  
train_img_list, test_img_list = train_test_split(img_list, train_size = 0.6)  
train_img_list, val_img_list = train_test_split(train_img_list, train_size = 0.6)
```



이름	유형	크기
img_list	list	984 items
test_img_list	list	394 items
train_img_list	list	354 items
val_img_list	list	236 items

■ 실습 데이터 준비

- 나눈 train, validation set의 파일명을 각각 text 파일로 저장

```
with open('data/train.txt', 'w') as f:
    f.write('\n'.join(train_img_list) + '\n')

with open('data/val.txt', 'w') as f:
    f.write('\n'.join(val_img_list) + '\n')
```

- yaml 파일에 나눈 데이터셋 다시 쓰기

 yaml : 사람이 읽을 수 있게 만들어진 데이터 직렬화 양식

```
import yaml
import os

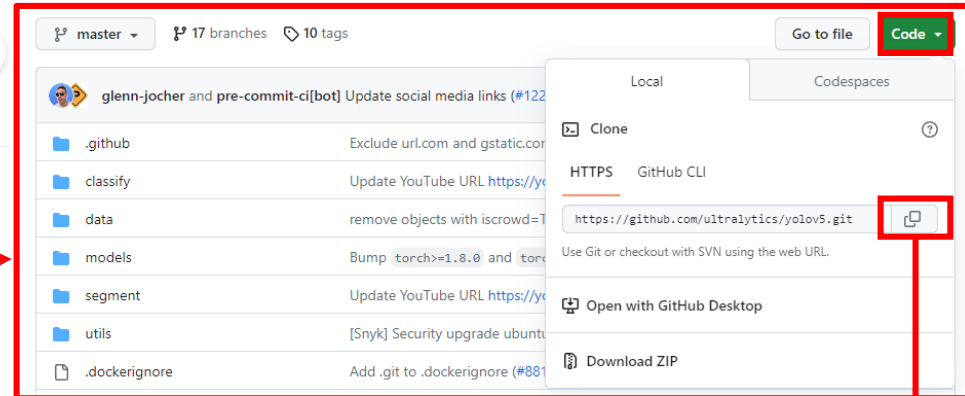
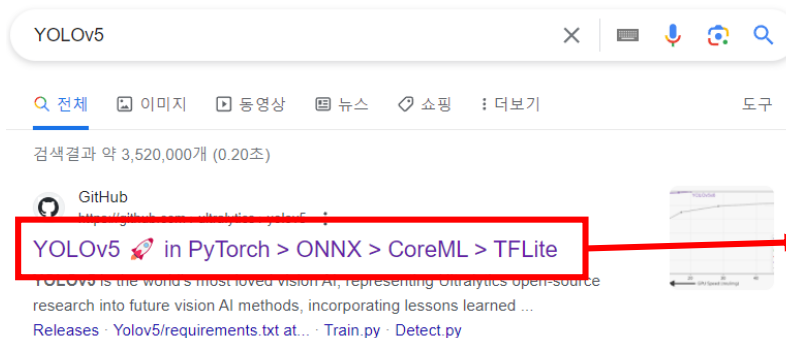
abs_train = os.path.abspath('data/train.txt') #절대경로 파악
abs_val = os.path.abspath('data/val.txt')
with open("data.yaml", 'r') as f:
    data = yaml.full_load(f) #yaml 파일로드
#data 변수에 추가
data['train'] = abs_train
data['val'] = abs_val
data['nc'] = 10
data['names'] = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
with open("data.yaml", 'w') as f:
    yaml.dump(data, f)
```

YOLOv5 실습



■ YOLOv5 실습

- google에 YOLOv5 검색 이후 GitHub 진입



```
!git clone https://github.com/ultralytics/yolov5.git  
!pip install -r yolov5/requirements.txt
```

YOLOv5 제작자가 작성해놓은 필요한 라이브러리 리스트
해당 text 파일을 읽으면서 자동으로 install

■ YOLOv5 실습

• YOLOv5 학습

```
!python yolov5/train.py --img 256 --batch 16 --epoch 50 --data data.yaml --cfg yolov5/models/yolov5m.yaml
```

<참고>

img: 이미지를 해당 크기로 resize

batch: 배치 사이즈 설정

epochs: 학습할 epoch 설정

data: 학습할 데이터에 대한 yaml파일

cfg: 모델의 구조

Pretrained Checkpoints ↗								
Model	size (pixels)	mAP ^{val} 50-95	mAP ^{val} 50	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1
YOLOv5x	640	50.7	68.9	766	12.1	4.8	86.7	205.7

다양한 모델 구조



Results saved to **yolov5/runs/train/exp12**

학습이 완료되면 학습 결과가 저장된 경로 출력

■ YOLOv5 정확도 지수

- YOLOv5 mAP

Epoch	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size
66/99	1.48G	0.02165	0.0209	0.01958	16	256: 100% 34/34 [00:09<00:00, 3.74it/s]
	Class	Images	Instances	P	R	mAP50 mAP50-95: 100% 2/2 [00:01<00:00, 1.49it/s]
	all	59	236	0.831	0.968	0.919 0.763

- Average Precision (AP)

- Recall, Precision 등을 활용하여 만든 인식 알고리즘의 정확도 지수

- mean Average Precision (mAP)

- 모든 class에 대한 AP의 평균

- mAP50

- 예측값과 라벨의 IoU의 임계값이 0.5 이상인 경우의 AP로 계산

- mAP50-95

- 예측값과 라벨의 IoU의 임계값이 0.5~0.95 이상인 경우의 AP의 평균으로 계산 (0.05 단위로)

■ 학습 결과 확인

- 학습 결과가 저장된 경로에 진입
- 저장된 weights 확인 가능
- best.pt : 학습 중 가장 정확도가 높았던 weights
- last.pt : 가장 마지막 epoch의 weights

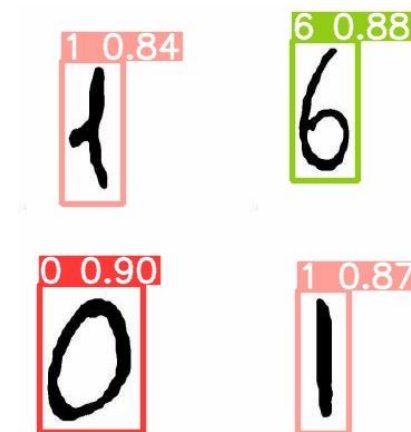


```
test_img_path = test_img_list[0]
!python yolov5/detect.py --weights Weight파일 경로 --img 256 --source "{test_img_path}"
```

detect.py : 예측 코드

Results saved to runs/detect/exp

위 셀을 실행하면 예측 결과가 저장된 경로 출력



예상 결과

각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #1) YOLOv5 실습

- 여러 하이퍼 파라미터를 조정하며 최적화
- 학습에 사용할 data 개수를 자유롭게 설정하며 학습 진행
- 다양한 모델 사용해보며 다른 점 및 결과 분석(xlarge, large, middle등)
- 예측 결과를 R-CNN의 결과와 비교 분석

실습 #2) keras를 활용한 YOLOv1 구현 (선택)

- keras를 활용하여 이론에서 배운 YOLOv1 구현
- 구현 성공 시, 아주 높은 과제 추가 점수 부여 → 예측 결과 및 분석과 함께 보고서 작성
- 구현 실패한 경우에도 구현 과정과 방법을 설명하며 실패한 이유 분석 시, 추가 점수 부여 가능 (단, 코드만 붙인 경우 X)